

Reinforcement Learning in SpaceRace

Task 1 – Basic DQN Implementation

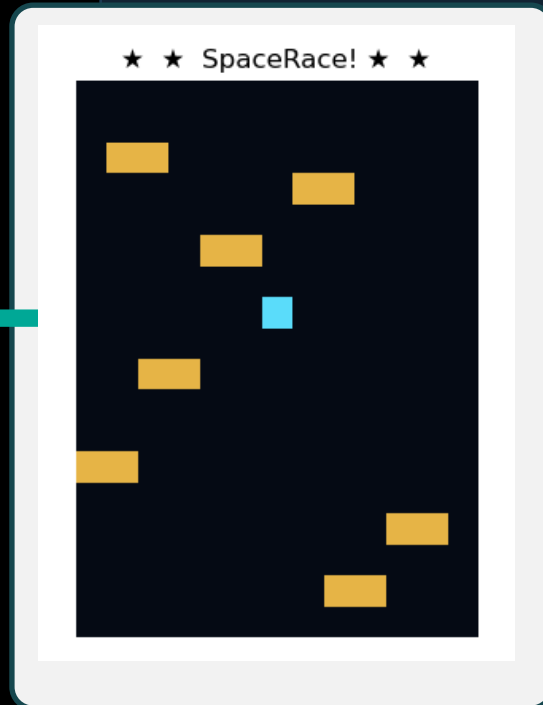
Task 2 – (TODO)

Task 3 – (TODO)

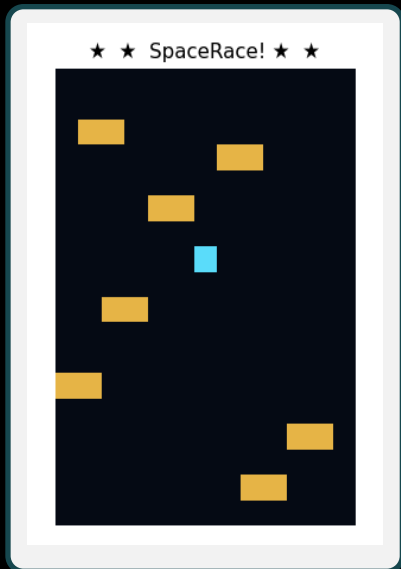
70698 – Francisco Silva

73208 – Rúben Silva

73554 – Miguel Cangalhas



Task 1 Agenda



- Environment Overview
- Neural Network Architecture
- Training Setup (Loss, Preprocessing, Loop)
- Heuristic Policy Design
- Ablation Studies (Architecture · Input · Frames · Regularization...)
- Final Configuration & Results
- Limitations & What's Next

Environment Overview

SpaceRace Game Context

- **SpaceRace:** Steer ship upward, avoid horizontal debris, 60s per round
- **Observation:** RGB frame (54 x 39 x 3), uint8
- **Actions:** 0 = Up, 1 = Down
- **Rewards:** +1.0 top crossing, -0.25 collision, +0.02 up, -0.01 down
- Game doesn't end on ship collision, only after stablished amount of time has passed

Difficulty Levels:

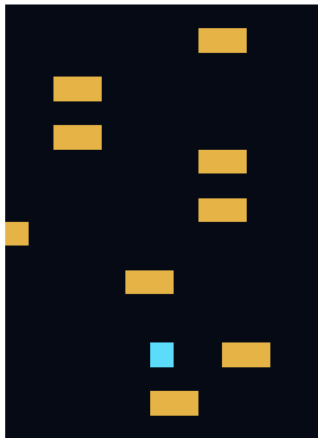
0: Deterministic debris,
normal speed

1: Faster debris

2: Random init positions

3: Dense + random debris

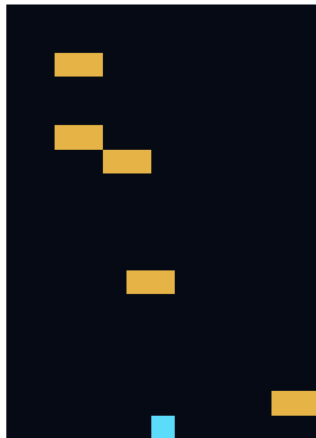
Difficulty 0



Difficulty 1



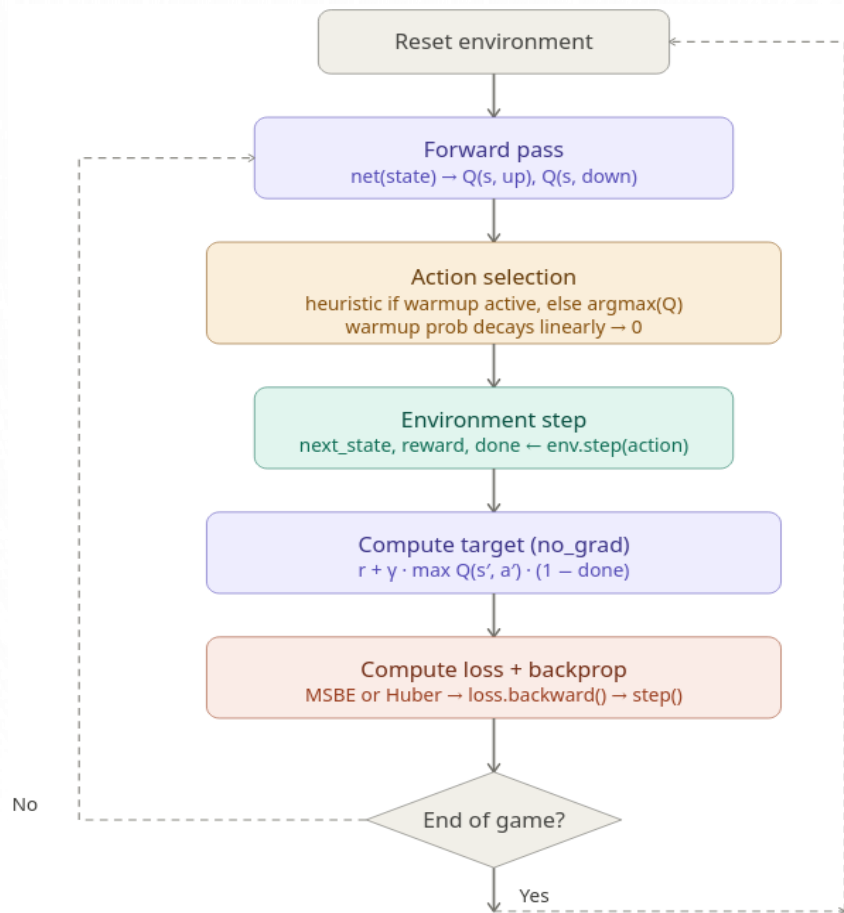
Difficulty 2



Difficulty 3



Training Loop



- **Action selection:**
heuristic if $\text{rand} < \text{warmup_pct}$, else $\text{argmax}(Q)$
- **Target:**
 $r + \gamma \cdot \max Q(s', a') \cdot (1 - \text{done})$, computed with `no_grad`
- **Loss:**
MSE or Huber $\rightarrow \text{loss.backward}() \rightarrow \text{clip_grad_norm}(\text{params}, 1.0)$
- **No replay buffer, no target network in Task 1**
 $\rightarrow$ intentional instability exposure

Evaluation

Setup: What to Measure?

Score

Count how many times the agent surpassed the top row (finish line); main indicator on how successful it was

Q-values

Use the mean of the Q-values to measure the agent's confidence; interpreted along score to determine if the agent's confidence lines up with its score

Reference Scores

Baseline Policies (no DQN): mean +- std

Difficulty	Random	Always Up
0	0.07 +- 0.29	17.00 +- 0.29
1	0.19 +- 0.42	17.00 +- 0.29

Any trained agent must exceed Always Up (~17) to be meaningful.

(Some) CNN Architectures

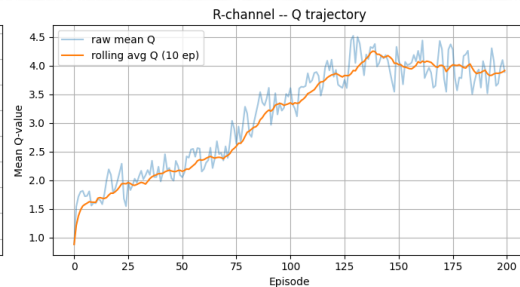
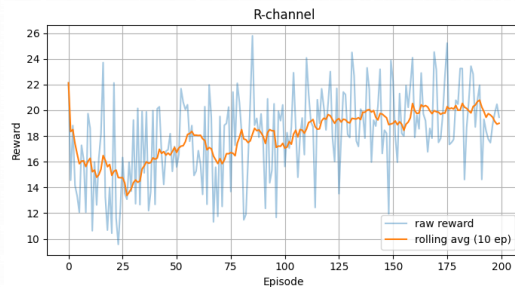
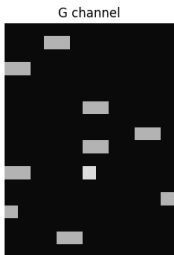
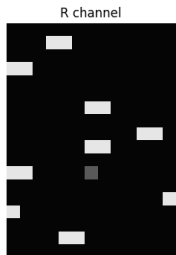
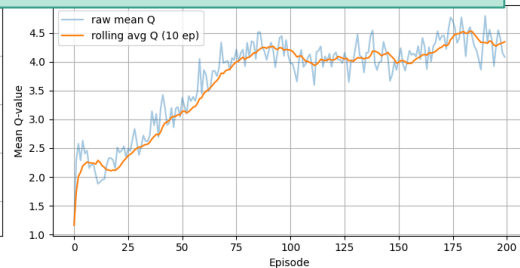
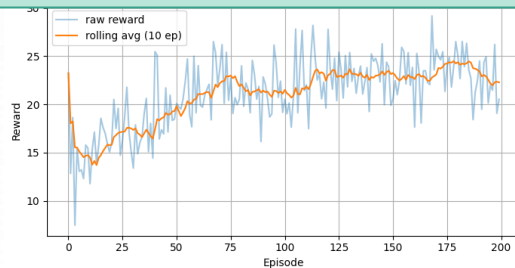
ID	Name	Model	Optimizer	Purpose
A	Simple	CNN(conv=[16,32], pool=MaxPool2x2, fc=[128->2], activation=ReLU)	Adam lr=5e-4	Baseline - compare performance with other existing architecture
B	Simple_rms★	CNN(conv=[16,32], pool=MaxPool2x2, fc=[128->2], activation=ReLU)	RMSProp lr=5e-4, alpha=0.99, eps=1e-8	Evaluate the effects of a standard optimizer for DQN
C, D	Simple_rms_dr op	CNN(conv=[16,32], pool=MaxPool2x2, fc=[128->2], activation=ReLU, dropout)	RMSProp lr=5e-4, alpha=0.99, eps=1e-8	Evaluate the effects of dropout (to “simulate” exploration)
E	Bigger	CNN(conv=[32,64,64], pool=AvgPool4x4, fc=[1024->512->2], activation=LeakyReLU(0.01))	Adam lr=5e-4	Exploration of how more convolution and pooling layers affect results
F	Bigger_rmsprop	CNN(conv=[32,64,64], pool=AvgPool4x4, fc=[1024->512->2], activation=LeakyReLU(0.01))	RMSProp lr=5e-4, alpha=0.99, eps=1e-8	Checking if RMSProp improves BiggerDQN results, as it did with SimpleDQN

In some runs the training rewards stagnate at 0, and in others they stagnate higher (~20) with a very low number of episodes (10). The RMSProp optimizer improved this due to the lack of Momentum which is hurtful in the case of a simple environment and DQN like our case.

The environment is simple (54x39x3)! There is no need to test a large amount of CNN, since smaller, simpler architectures will probably outperform deeper, more complex ones (as confirmed by model F’s performance, which is worse and takes way longer). Thus, we did some deep testing on model B.

Ablation Study - Input Channels

3-Channel RGB vs R-Channel Only



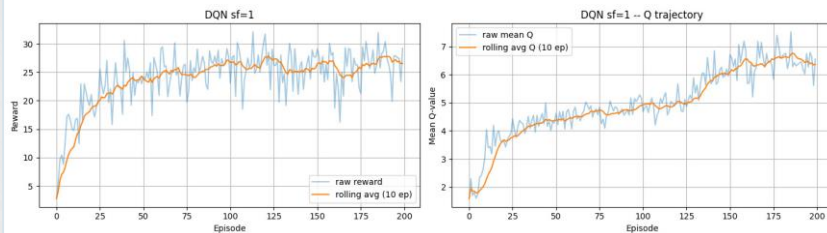
Winner: RGB (3ch) • RGB: 24.0 vs R-Only: 12.0 (50% drop) • Q-values: 4.04 vs 3.38 • R-channel loses inter-channel feature redundancy needed for depth and motion perception; R is also more unstable in terms of results obtained in different training runs

Ablation Study - Frame Stacking

With one single static frame, the model cannot infer the speed of the debris that are appearing; stacking multiple frames can help with this. Even though the replay buffer is not intended for this task and this strategy is best coupled with it, we still tested it, and the results improved for 4 frames, and maintained for 2. We expect an even bigger improvement on this in task 2 with the memory buffer where each state will be a composition of multiple frames still satisfy the Markov property and can induce speeds, with i.i.d. training data

2 Stacked Frames

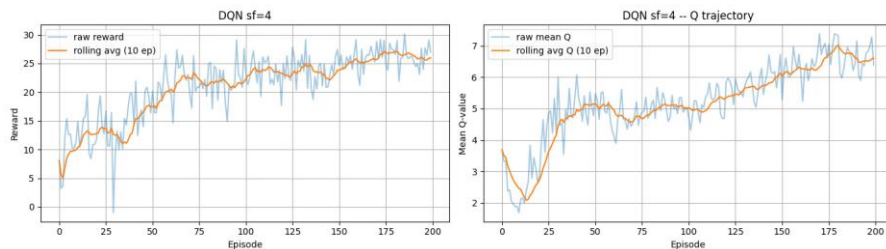
Difficulty = 0



DQN n_frames=2: score=20.00+-0.00 q=6.5118

4 Stacked Frames ★

Difficulty = 0



DQN n_frames=4: score=27.00+-0.00 q=6.6740

Note: 1 stacked frame (== no stacked frames) results are previous slide's RGB results

Ablation Study - Frame Stacking

1 vs 2 vs 4 frames

Ablation	Tested	Winner	Key Finding
Frame Stacking	1 / 2 / 4 frames DQN	4 frames	4-frame sweet spot: motion awareness slightly more complexity overhead

Key Results

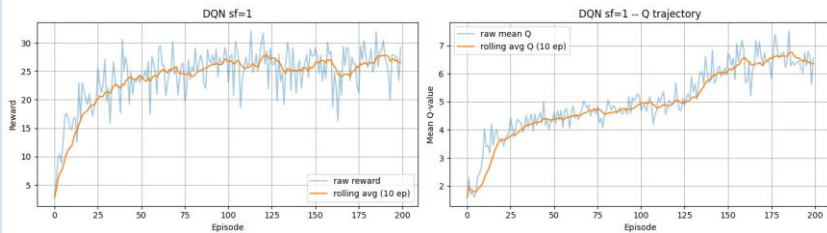
- DQN (4 frames): Best score (27.0) and highest Q-value ($q = 6.67$) - strongest policy confidence
- DQN (2 frames) maintained
- 4 frames = sweet spot: adds motion/velocity awareness without overwhelming the model with more frames than needed (e.g. 8)

Ablation Study - Dropout & BatchNorm

In the context of the DQN without exploration strategy (our case), the model is greedy: once it finds a good enough policy it sticks to that, without trying other actions by exploration. Our idea with dropout was that, since it randomly drops neurons, it might occasionally flip the Q-Values in a way that another action which is not the best is picked. However, as can be seen below, the results for reward convergence were not better. Our conclusion was that this basic DQN already has a hard time learning these simple deterministic environments, and introducing this additional signal variance causes unnecessary instability.

Dropout (0.1)

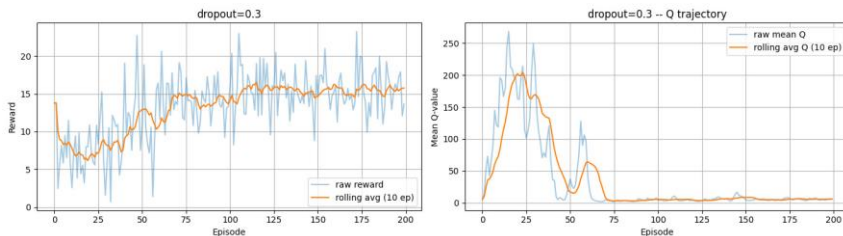
Difficulty = 0



dropout=0.1 score=20.00+-0.00 q=6.0963

Dropout (0.3)

Difficulty = 0

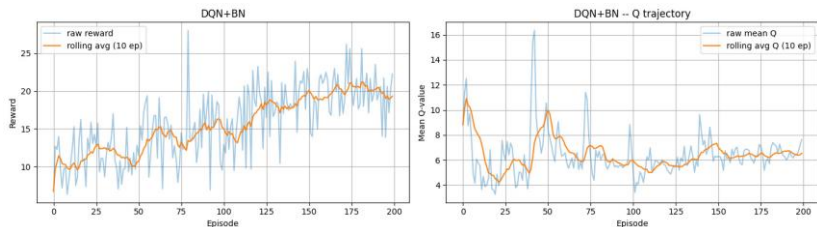


dropout=0.3 score=17.00+-0.00 q=5.9204

Ablation Study - Dropout & BatchNorm

Dropout acts as a learning tax in online RL; agent already struggles with moving target
BatchNorm also hurts performance

Batch Normalization



BN model: score=24.00+-0.00 q=7.1547

- **Dropout 0.0: Score 27.0, Q-val 6.67 - Winner**
- Dropout 0.1: Score 20.0 - Worse
- Dropout 0.3: Score 17.0 - Worse
- BatchNorm: Score 24.0, Q-val 7.15 - Higher confidence with lower performance

Key insight:

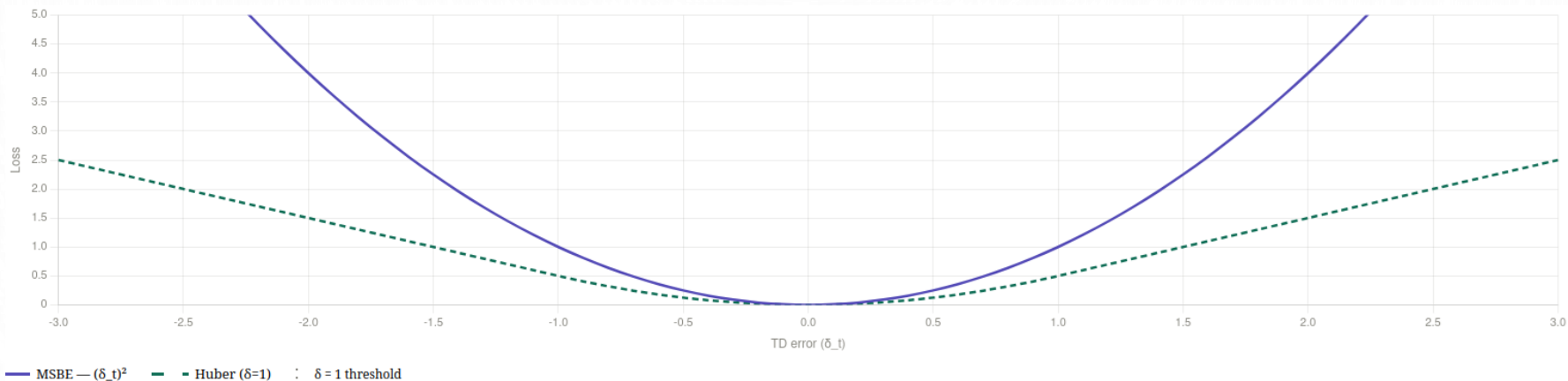
- Dropout is a “learning tax” in online RL;
- BatchNorm is worse and overconfident

Ablation Study - Q-Learning Loss: MSE vs Huber

MSE: Quadratic for all errors (gradients can explode on large errors)

Huber: Quadratic for small errors, linear for large (bounded gradients; good when outliers)

Reward clipping keeps TD targets small but loses reward magnitude info.



Ablation Results

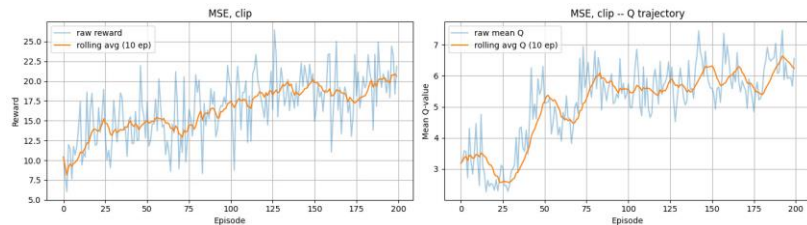
- **Huber, and clip rewards [-1, 1]: 27.0** - Winner. Similar learning signal, better stability with gradient clipping as safety net.
- **MSE, and clip rewards [-1, 1]: 26.0** - Worse. Less stable.

Ablation Study - Q-Learning Loss: MSBE vs Huber

Despite the errors not having big magnitudes in this scenario, Huber acts as a safety net along reward clipping, seen in the stability of the Q-values trajectory and in the loss standard deviation being much smaller than in MSE. We also deem that Huber will have an even better effect on difficulties 2 and 3, since there are more outliers in those cases due to them not being deterministic environments like difficulty 0 and 1

MSE

Difficulty = 0

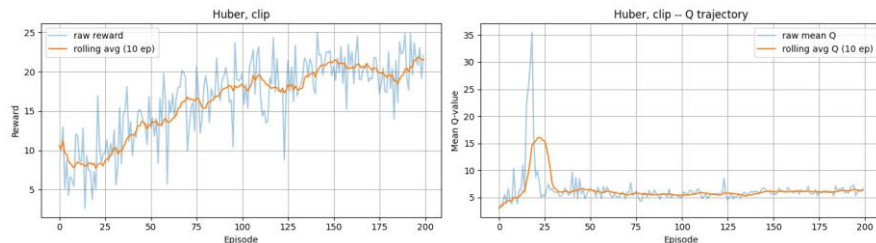


MSE, clip

score=26.00+-0.00 loss_std=0.1510

Huber

Difficulty = 0



Huber, clip

score=27.00+-0.00 loss_std=0.0179

In the baseline difficulty we can see that in Huber, the model achieves similar rewards as MSE, but its Q-values are much more stable; the model's confidence is more realistic. We also noticed that Huber's performance was way more stable than regular MSBE

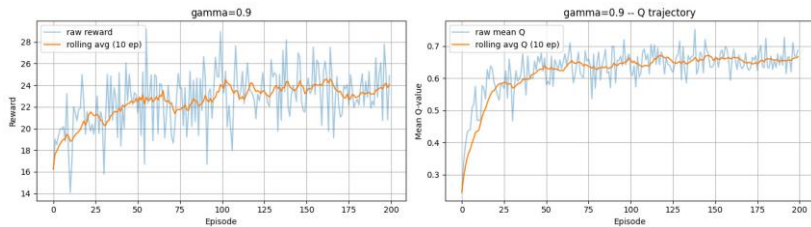
Due to this, Huber will be adopted in our model's training and evaluation processes

Ablation Study - Gamma

Here we will test how far into the future our agent should “look” in order to optimize its path; we tested gamma values 0.90 and 0.95 (aswell as 0.99 which was used in previous slides)

Gamma (0.90)

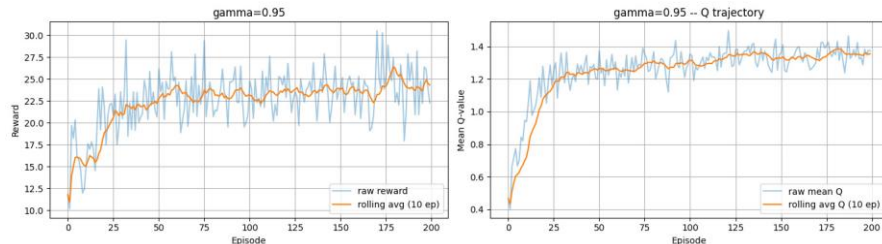
Difficulty = 0



gamma=0.9 score=26.00+-0.00 q=0.5584 q_growth=2.82x

Gamma (0.95)

Difficulty = 0



gamma=0.95 score=26.00+-0.00 q=1.2792 q_growth=2.95x

Decreasing the gamma value did not improve the results; the top row (far reward) should be its main priority

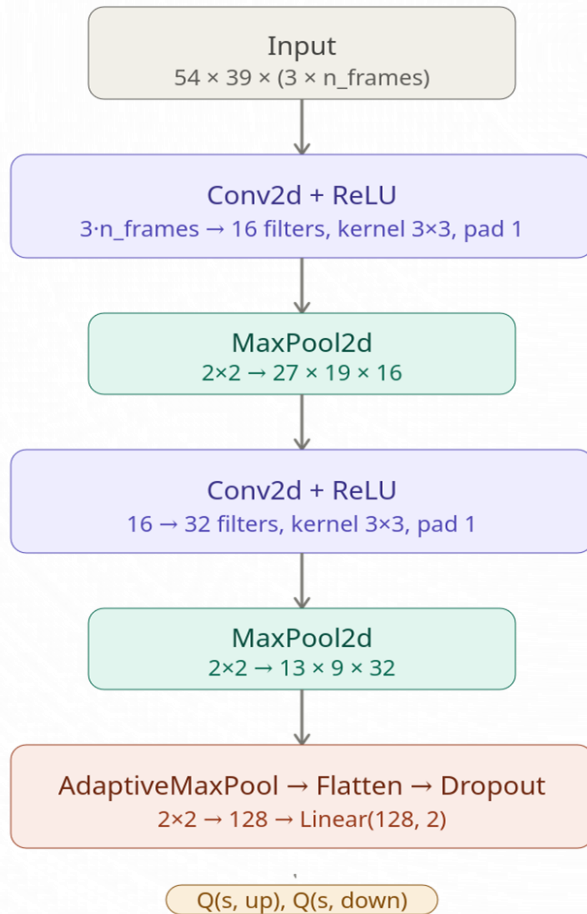
Final Configuration

Chosen Hyperparameters for Submission

Hyperparameter	Final Choice	Reason
Architecture	Simple_rms	Simple architecture provides better performance
Input	RGB (3ch)	Feature redundancy + depth perception
Frames	4 stacked	Motion awareness without overhead
Optimizer	RMSprop (lr=5e-4, a=0.99)	Stable in non-stationary data
Loss	Huber	More stable than regular MSE
Gamma	0.99	Tight feedback loop
Dropout / BN	Dropout 0.0 / No BN	Deterministic policy, fastest learning; BN fails with batch_size=1
Warmup pct	0.7	Curriculum: strong guidance

CNN Best Model (B_CNN)

Architecture:



Key Design Choices

- 3×3 kernels resemble ship and debris physical size
- Progressive channels: $3 \rightarrow 16 \rightarrow 32$ (abstract representations)
- Single FC layer ($128 \rightarrow 2$): sufficient for 2 actions
- No BatchNorm: batch size=1, statistics too noisy
- **ReLU** (small net $\rightarrow$ dying neurons not a risk)
- **RMSprop** over Adam: no Momentum $\rightarrow$ better for non-stationary data

CNN Best Model (B_CNN)

Architecture:

Feature Extraction

- Two conv layers with progressive channel expansion (3->16->32) — forces the model to build increasingly abstract representations, from edges and colors in layer 1 to shapes like the debris and ship in layer 2 (for example)
- 3x3 kernels match the physical size of the ship and debris (or part of them), making them well-suited to detect the key elements in the scene
- Single linear layer (128->2) is sufficient; with only 2 actions and a spatially simple environment, a deep classifier adds no value
- Better than:
 - Wider (16->64): overkill for a 3-color env with 2 actions, more parameters = slower training with no benefit
 - Deeper (3+ layers): risk of over-shrinking the small 54x39 input before meaningful features are extracted

ReLU + RMSProp

- **ReLU**

ReLU is a solid activation function for this case; our network is small enough that the problem of dying neurons will not pose a problem
- **RMSProp**

Since we are not using a replay buffer, our data is highly correlated (thus not i.i.d.). Because of this, and due to the fact that it is non-stationary (distribution of states the agent visits changes as the policy improves, so early and late training data come from different distributions), Momentum hurts as target optimal parameters shift over time. RMSProp is better than Adam since it lacks the Momentum part. It improved stability.

Heuristics: Design & Comparison

4 Different Training Cases for Heuristic Usage

Random

Execute actions randomly; no real usage, just used for performance comparison

DQN (No heuristic)

Use a DQN to calculate the Q-values and learn without an initial heuristic in mind; initial actions are chosen randomly since the network has nothing to guide itself with

Our Focus

```
graph TD; Focus[Our Focus] --> Full[DQN (Full heuristic)]; Focus --> Warmup[DQN (Warmup with heuristic)]; Warmup --> NoHeuristic[DQN (No heuristic)];
```

DQN (Full heuristic)

Use a DQN to calculate the Q-values, but execute actions by strictly following a specific heuristic. The model essentially learns the heuristic; good for deterministic environments like difficulty 0 and 1, but may not be as good when debris are randomized (difficulty ≥ 2)

DQN (Warmup with heuristic)

Use a DQN to calculate the Q-values and, initially, execute actions by following an heuristic warmup_pct% of the time (controlled by warmup_pct parameter). Better for all types of environments

Heuristics

Can a better developed heuristic improve the results and make the model learn?

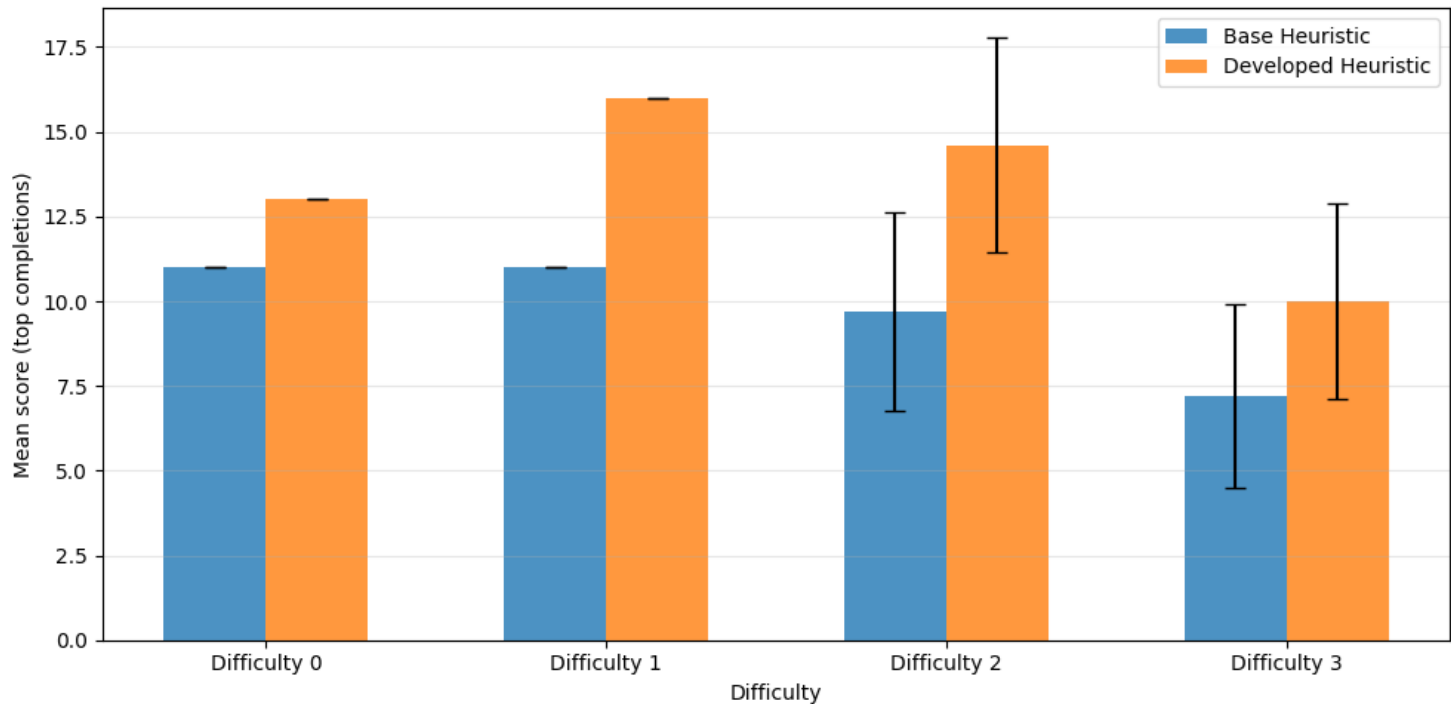
Simple Heuristic

If there are no debris upwards, go up. Otherwise,
go down

(Main) Developed Heuristic

Scores both directions (up and down) by scanning the next 3 rows in each direction. For each row it checks a window of columns to the left of the ship where closer rows are weighted more heavily than farther ones. Clear rows add weighted points, rows with debris subtract a heavy penalty. A small upward bias breaks ties in favor of going up since reaching the top scores points. The direction with the higher score is picked

Heuristics: Design & Comparison



In full heuristic play, our developed heuristic is clearly superior to the simplistic heuristic (base) presented before. The base heuristic performs okay in the easiest case (difficulty 0), but as difficulty increases, the gap between both heuristics widens. So, we will stick with the developed heuristic to improve the model's performance

Heuristics: Design & Comparison

2 Different Training Cases for Difficulties

Our Focus

No difficulty warmup

Train the model on the specific difficulty we're trying to evaluate the performance for.

Note: from our tests, the results are not as good for difficulty 1, so we will stick with the other option

Difficulty Warmup

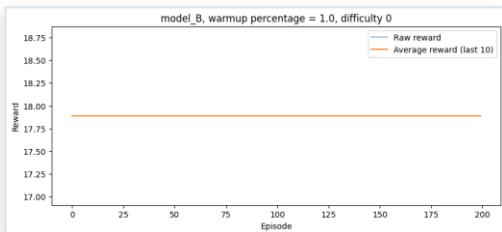
Warmup the model by training on difficulty 0 (baseline) and then train on difficulty 1; essentially, the model learns with difficulty 0 so that it can start on a better performance on the next task. Use heuristic warmup on difficulty 0; do not use heuristic on difficulty 1. This provided us with better results

Full Heuristic vs. Warmup

We will compare how the model performs by either always having the heuristic choosing the action (learns to imitate it), or with a probability equal to warmup_pct; the heuristic acts as a helper for the net's total rewards to converge faster. Here, we also trained on difficulty 1

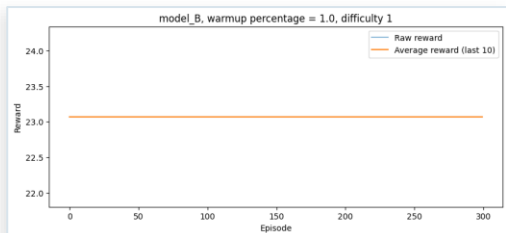
Full Heuristic

Warmup (difficulty = 0)



Score: 15.00 +- 0.00 | Q-value: 5.1132

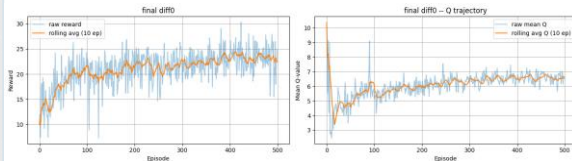
Faster (difficulty = 1)



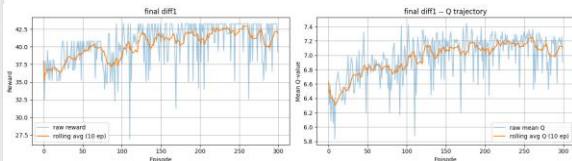
Score: 6.00 +- 0.00 | Q-value: 7.0972

Warmup (0.7) ★

Warmup (difficulty = 0)



Faster (difficulty = 1)



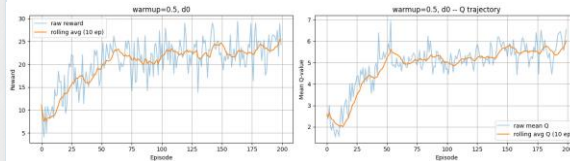
Eval diff0: 27.000 +- 0.000

Eval diff1: 32.000 +- 0.000

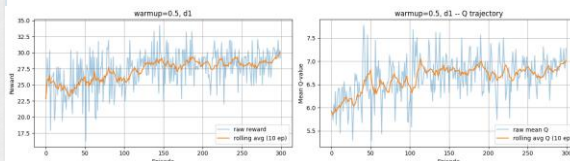
Major improvement over not using an heuristic or strictly imitating one!

Warmup (0.5)

Warmup (difficulty = 0)



Faster (difficulty = 1)



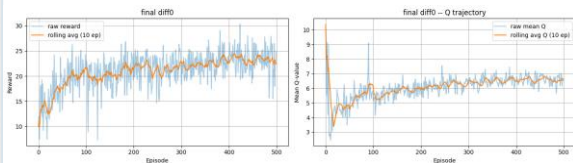
warmup=0.5 diff0=0.00+-0.00 diff1=27.00+-0.00

Full Heuristic vs. Warmup

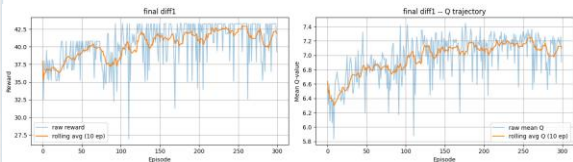
We chose warmup_pct equal to 0.7 for the following reasons:

Warmup (0.7) ★

Warmup (difficulty = 0)



Faster (difficulty = 1)



Eval diff0: 27.000 +- 0.000

Eval diff1: 32.000 +- 0.000

Major improvement over not using an heuristic or strictly imitating one!

The model with warmup set to 0.7 was the one that achieved better results.

Full heuristic: the model learns the heuristic but it fails to obtain a good enough score, in particular for difficulty 1, despite its mean Q-value being very large (overconfident)

Warmup 0.7: having the model imitate the heuristic for some iterations, while also allowing it to learn on its own most of the time, was a major improvement over just fully using the heuristic. It stabilizes rather fast and improved a lot over not using an heuristic or imitating one

Warmup 0.5: also an improvement over full heuristic, but not as good as warmup 0.7 (e.g. more unstable, look at score of difficulty 0 where it obtained 0)

Training Curriculum

Progressive Difficulty Strategy

Phase 0 - Difficulty 0

500 episodes

- Warmup pct = 0.7 (heuristic guides 70% of steps)
- Agent learns basics: debris avoidance, upward bias
- Stabilises around episode 150



*Same model weights
continue*

Phase 1 - Difficulty 1

300 episodes (same model)

- NO heuristic - diff0 acts as an initial helper/warmup
- Faster debris
- Achieves competitive diff1 performance

Catastrophic Forgetting & Instability

There is a fundamental problem in not using a replay buffer: the training data is not i.i.d. (training data is highly correlated); this can cause catastrophic forgetting, where the model tends to forget things that it learned in iterations near the beginning of training, and is biased to things that happened later. What does this fundamentally result in?

Training

The model has an okay performance in training without the memory buffer and target network implementations. It achieves moderate rewards in both difficulty 0 and difficulty 1 separately.

Evaluation

However, when evaluating, the performance of the model in difficulty 0 is worse than in difficulty 1 (after training in 1). Should this happen? Probably not: difficulty 0 is the easiest, why would the model perform worse than in difficulty 1? Simple; the model forgot what it learned when training in difficulty 0. This will be fixed in task 2 through the implementation of the memory buffer (and the help of stacked frames to infer velocities)!

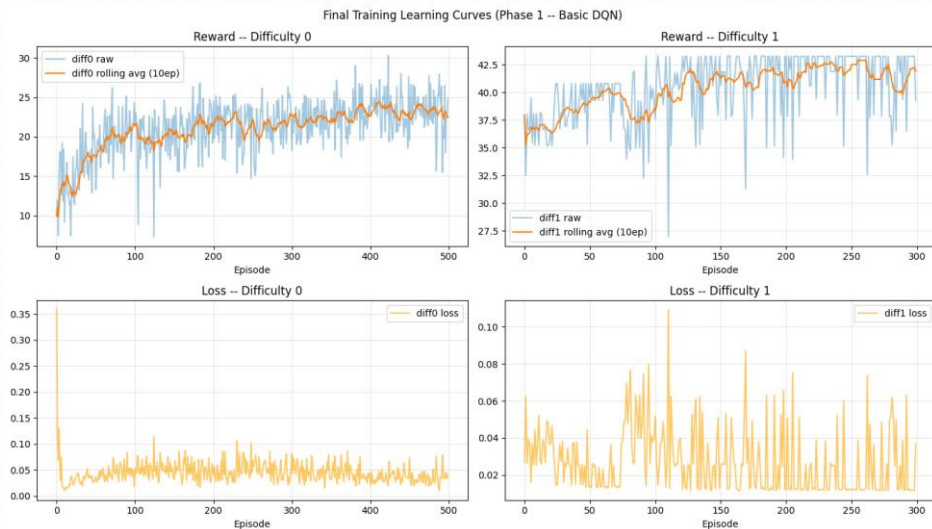
Performance on difficulty 0 after training on difficulty 1, only using heuristic on 0: `Eval diff0: 20.000 +- 0.000`

Performance on difficulty 0 after training on difficulty 1, heuristic on both: `Eval diff0: 8.000 +- 0.000`

Note that this overfitting can be minimized by choosing very good parameters, however, it is way harder than with a proper memory buffer + target network DQN

Final Results

Best Configurations Across Difficulties



Difficulty 0 (Huber) - Score: 27.00

Difficulty 1 (Huber) - Score: 32.00

Diff 0 (Huber): 27.00 +- 0.00 | Diff 1 (Huber): 32.00 +- 0.00 | Stabilisation: ~ep 150 | Baseline (Always Up): 17.0 - DQN with heuristic and difficulty warmups surpasses!

Problems & What's Next

Three key issues identified in Task 1 - all addressed in subsequent tasks:

Without replay buffer or target network, training is unstable and non-reproducible. Results vary wildly between runs.

- No memory buffer → Highly correlated data; training diverges depending on early iterations → Fixed in Task 2
- No target network → Target computed by same network being updated; chasing a moving target → Fixed in Task 2
- No exploration strategy → Greedy: once a good policy is found, never searches for better ones → Fixed in Task 3

Reinforce Learning in SpaceRace

TASK 1 - Basic DQN Implementation

70698 – Francisco Silva
73208 – Rúben Silva
73554 – Miguel Cangalhas

THANK

YOU !